Planning with Monte Carlo tree search

Imagine many endings, then choose the first step

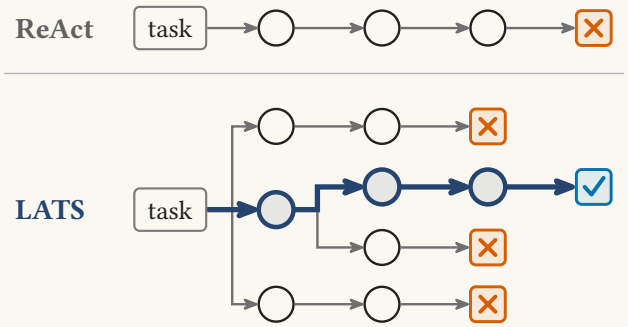
Carlos Cotrini

ETH Zurich

October 2026

Language Agent Tree Search (LATS)

LATS lets a language-model agent explore many lines, not one.



Zhou et al., ICML 2024 (PMLR 235).

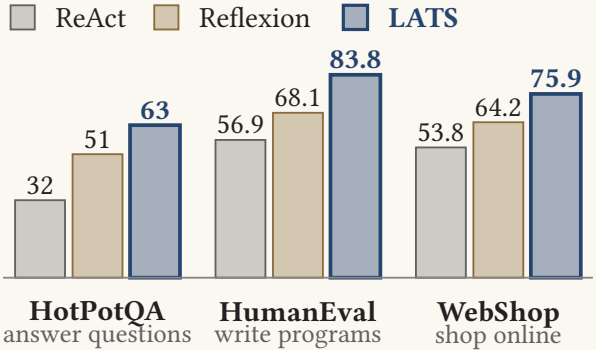
1 A plain agent, ReAct, follows one line of steps, with no way back.

2 LATS tries several steps at each fork.

3 It searches that tree by **Monte Carlo tree search**.

What the tree buys

The same model, GPT-3.5, scores higher when it searches.



1 ReAct follows one line of steps; Reflexion retries, with notes on its failures.

2 LATS searches a tree of lines and leads on all three.

3 The price: more computation per task.

Zhou et al., ICML 2024, Tables 3, 4 and 6. GPT-3.5; scores out of 100.

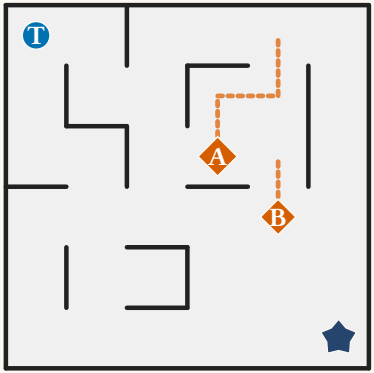
Imagine before you step

A labyrinth with two Medusas, and a habit that programs share.



Theseus and the two Medusas

Two Medusas patrol the labyrinth.



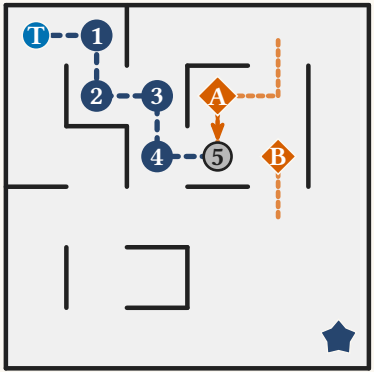
1 Theseus, **T**, wants the lair, the star.

2 Medusas **A** and **B** walk dotted rounds for ever.

3 Anyone in her cell, or the cell she enters next, turns to stone.

He can imagine the ending

A wrong step is final, but an imagined walk costs nothing.



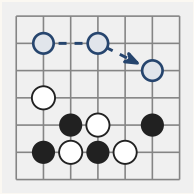
1 On paper the shortest route is 10 steps.

2 In his head he walks it, turn by turn.

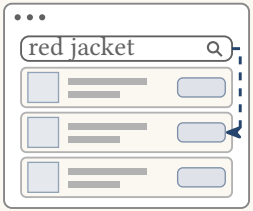
3 At turn 5 Medusa **A** reaches him: stone. Nothing is lost.

The same habit runs inside programs you know

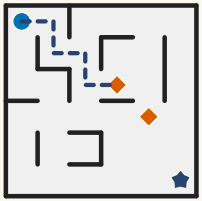
Some Go programs and web agents imagine many endings first.



a program that plays Go



a program that shops for you

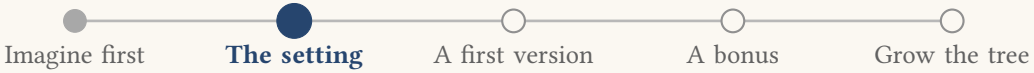


Theseus

Dashed: the endings each one imagines first.

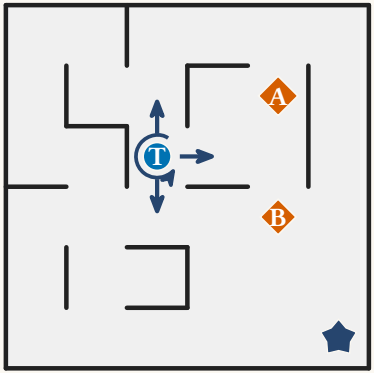
The setting

States, actions, returns and values, told on the labyrinth.



State and actions

State: a cell and a turn. Action: stay, or step.



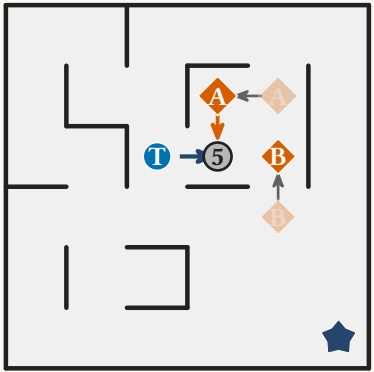
1 Here: the **T**'s cell, turn 4.

2 The Medusas keep a timetable: another turn, another state.

3 Here: four actions, stay or step up, right or down.

Transitions

Asking the world what follows an action costs nothing.



1 He asks about one action: step right.

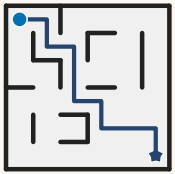
2 The answer: the next state, turn 5.

3 It also says if the run is over: stone.

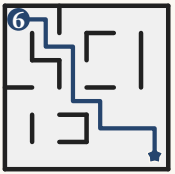
Reward and return

A run pays a return G, from 0 to 1: the sooner at the lair, the more.

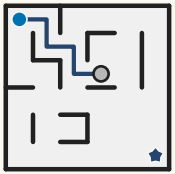
Reward: a turn pays 0, except reaching the lair on turn t: 0.97^t.
Return G**:** the rewards added up.



at the lair
on turn 10
$G = 0.97^{10} \approx 0.74$



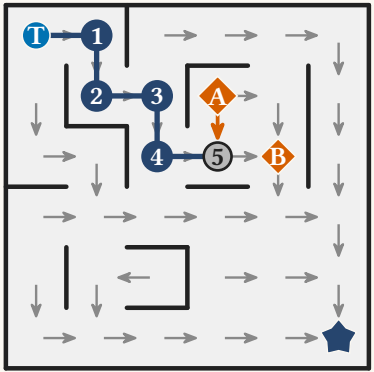
waits 6 turns, then at
the lair on turn 16
$G = 0.97^{16} \approx 0.61$



turned to stone
on turn 5
$G = 0$

Policy and trajectory

A policy picks an action in every state.



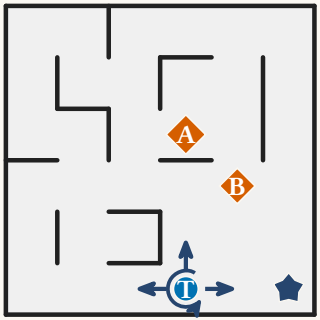
1 This one steps to the cell nearest the lair.

2 From the door it gives a trajectory.

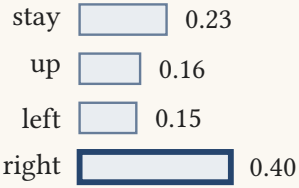
3 It ends in stone on turn 5: $G = 0$.

The goal

Choose the action with the largest average G.



average G of 10,000 random runs after each action



1 Per action: imagine many runs, average their G.

2 Play the largest: here, right.

3 No training, only simulation.

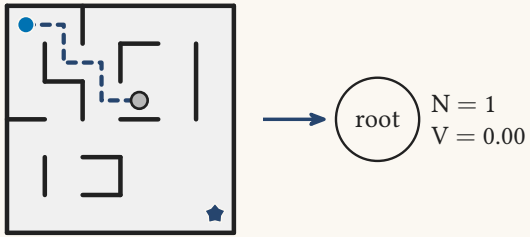
A first version

A tree of passes, and what one pass does.



A counter on every state

Each state carries N, its passes, and V, their average G.



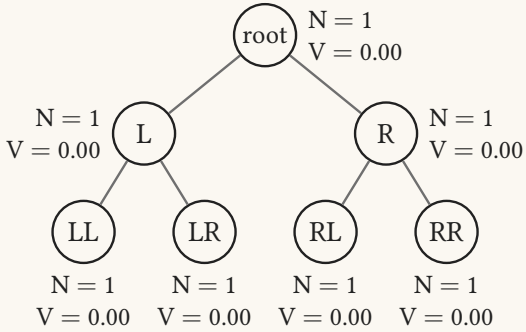
a state: the door, on turn 0

1 A pass is one imagined run through it, here ending in stone: $G = 0$.

2 Born with $N = 1$ and $V = 0$: one imagined visit that paid nothing.

A tree of depth d

The first version: a node for every state up to d steps ahead.



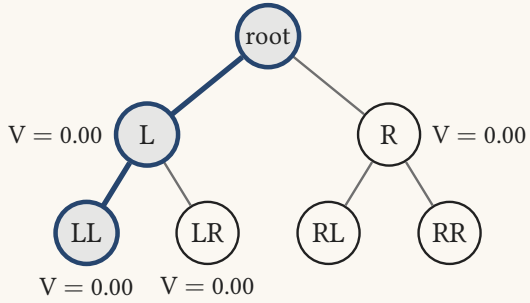
1 A toy labyrinth: three forks. The door is the root.

2 With $d = 2$: 7 nodes.

3 Each starts at $N = 1$, $V = 0$.

A pass, step 1: go down

A pass goes down, into the child with the largest V.

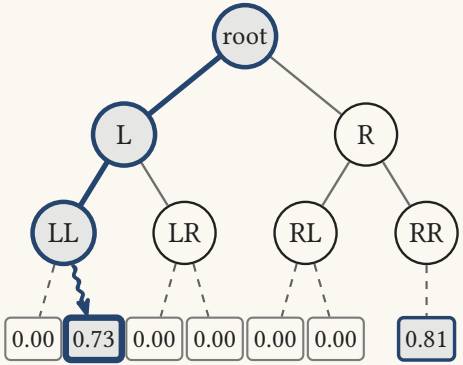


1 $V = 0$ twice: a tie, so the left child.

2 Again, and it stops at LL: the tree ends.

A pass, step 2: a rollout

A rollout plays at random to the end and reads off G.

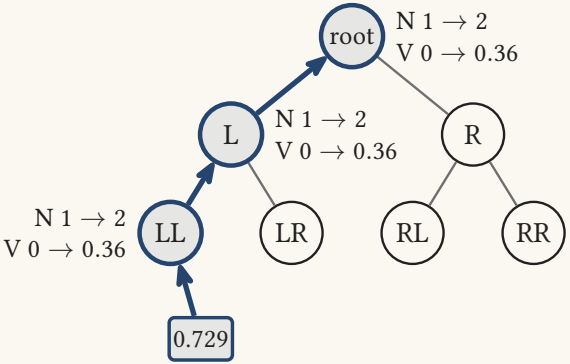


1 In the toy a lair on turn t pays 0.9^t, any other end 0.

2 $G = 0.9^3 = 0.729$. A rollout adds no nodes.

A pass, step 3: walk back up

Each node on the path counts the visit and moves V toward G.



1 $G = 0.729$ goes back up the path.

2 Each node: $N \leftarrow N + 1$, $V \leftarrow V + (G - V)/N$. At LL, $V = 0.00 + (0.729 - 0.00)/2 \approx 0.36$.

3 Same at L and the root; the others are untouched.

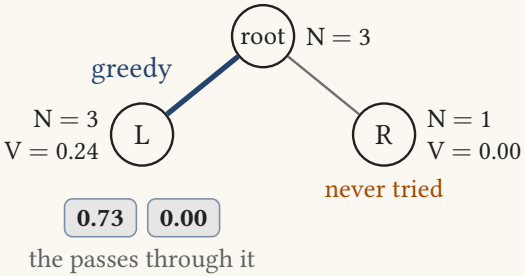
A bonus for the barely tried

Why the first version sticks, and how UCT repairs it.



A child that paid once

A child that paid once beats one nobody tried, and greedy never goes back.



1 Two passes so far, both through the left child.

2 Only a pass through the right could change its V; greedy sends none.

UCT

A pass goes into the child with the largest score.

UCT: Upper Confidence bounds applied to Trees.

$$\mathrm{UCT}(s) \;=\; V(s) \;+\; w \cdot \mathrm{bonus}(s)$$

$$\mathrm{bonus}(s) \;=\; \sqrt{\frac{\ln N(\mathrm{parent}(s))}{N(s)}}$$

ln: natural logarithm.

1. w is a weight we choose.
2. It is large when $N(s)$ is small next to its parent's.
3. Each pass that skips s raises it, so s gets its turn.

Only the descent changes.

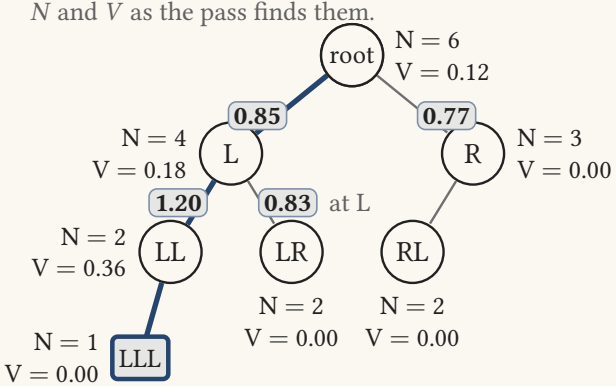
Grow the tree where passes go

One node a pass: select, expand, simulate, back up.



Six passes on the toy

Six passes grow the tree to 7 nodes, UCT at $w = 1$.

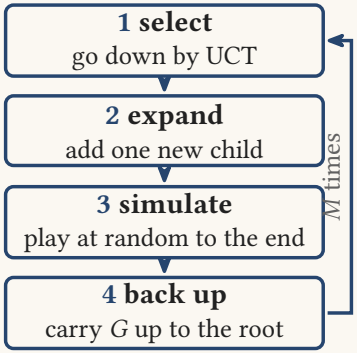


pass	adds	paid	UCT of L / R
1	L	0	*untried*
2	R	0	*untried*
3	LL	0.73	**0.74** / 0.74
4	LR	0	**0.92** / 0.83
5	RL	0	0.82 / **0.90**
6	LLL	0	**0.85** / 0.77

Pass 4 stops at L: LR untried.
Pass 5: R has the lower V but the higher UCT.

The four phases

Every pass: select, expand, simulate, back up.



`mcts_move(state, n_attempts, w, rng)`

```
root = Node(state, None)
for i in range(n_attempts):
1   leaf = descend(root, w)
2   leaf = expand(leaf)
3   r = rollout_reward(leaf.state)
4   backprop(leaf, r)
return most_visited_child(root)
```

M is `n_attempts` in the code.